HIGH-LEVEL DESIGN REPORT

GUIDE

(Guided User Itinerary & Destination Explorer)

QUAD-CORE Team

Ebrar Sude Doğan
Erdem Baran
Kayrahan Toprak Tosun
Tuna Kodal

TOBB University of Economics and Technology
Department of Computer Engineering
BIL 495 / YAP 495 Graduation Project

Version: 1.0
Date: January 2026

Contents

1 Introduction **2**
1.1 Purpose of the system 2
1.2 Design goals 2
1.3 Definitions, acronyms, and abbreviations 2
1.4 Overview 4

2 Current software architecture **4**

3 Proposed software architecture **5**
3.1 Overview 5
3.2 Subsystem decomposition 7
3.3 Hardware/software mapping 8
3.4 Persistent data management 9
3.5 Access control and security 10
3.6 Global software control 10
3.7 Boundary conditions 10

4 Subsystem services **10**
4.1 Presentation services (Web UI) 10
4.2 Application services 11
4.3 Data services (POI & media storage) 11
4.4 Security services (API protection and access control) 11
4.5 Integration services (routing and offline preparation) 11

5 Glossary **12**

References **12**

1 Introduction

This document presents the High-Level Design (HLD) of the GUIDE system. The purpose of this document is to describe the overall architectural structure of the system, identify its major subsystems, and outline their responsibilities and interactions at a conceptual level.
GUIDE (Guided User Itinerary & Destination Explorer) is designed as a software system that generates structured travel itineraries based on user preferences and predefined data sources. The HLD focuses on architectural decisions, subsystem boundaries, and design rationale, while implementation-level details are intentionally excluded.

1.1 Purpose of the system

The purpose of the GUIDE system is to provide a scalable, modular, and maintainable software platform that supports itinerary generation and route-oriented travel planning. The system is designed to deliver its core services reliably while allowing future functional and non-functional extensions without requiring major architectural changes.
This High-Level Design (HLD) document describes the overall architectural structure of the system, identifies its main subsystems, and explains their responsibilities and interactions. The document focuses on high-level design decisions and subsystem boundaries, and intentionally excludes low-level implementation and technology-specific details.

1.2 Design goals

The design goals define the architectural principles that guide the high-level design of the GUIDE system. These goals are derived from expected usage scenarios, system constraints, and long-term maintainability considerations. They serve as criteria for evaluating architectural decisions and subsystem decomposition.

The primary design goals are as follows:

- Modular and loosely coupled subsystems
- Maintainability and extensibility
- Security and controlled access
- Performance-awareness for typical usage (targeting responsive route generation)
- Reliability via isolation of heavy components and controlled dependencies

1.3 Definitions, acronyms, and abbreviations

The following acronyms and abbreviations are used throughout this document.

- **API:** Application Programming Interface
- **CDN:** Content Delivery Network
- **GDPR:** General Data Protection Regulation

- **HLD:** High-Level Design
- **KVKK:** Kişisel Verilerin Korunması Kanunu
- **OSM:** OpenStreetMap
- **OSRM:** Open Source Routing Machine
- **POI:** Point of Interest
- **REST:** Representational State Transfer
- **TTS:** Text-to-Speech
- **UI:** User Interface
- **UX:** User Experience

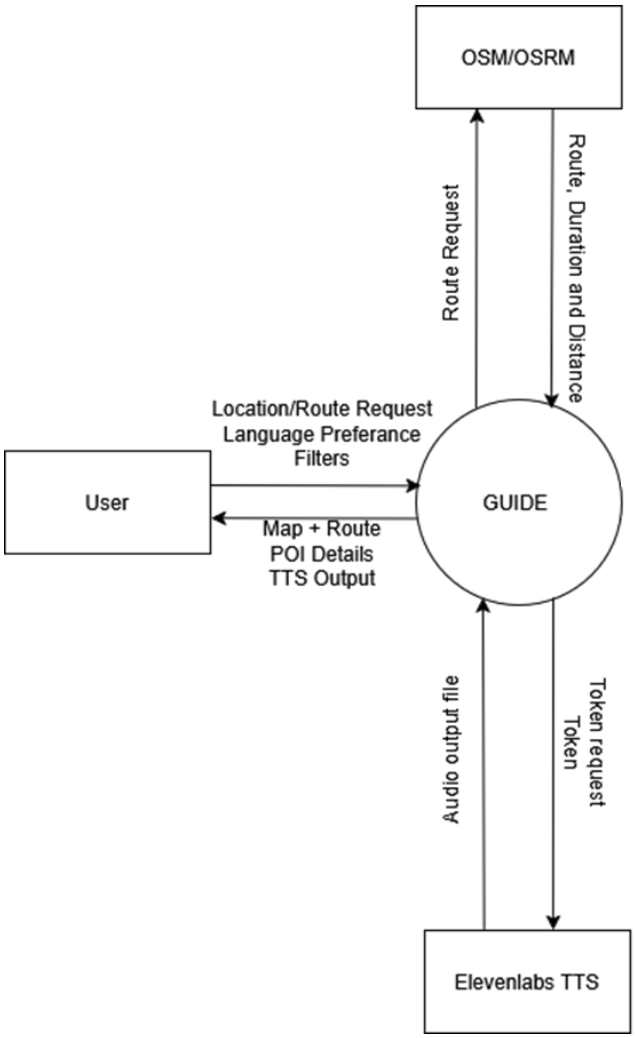

Figure 1: GUIDE system context diagram.

1.4 Overview

The diagram in Figure 1 illustrates the system boundary of GUIDE and the primary external entities that interact with it. This context diagram focuses exclusively on runtime interactions and external dependencies that directly affect system behavior, providing a clear and simplified view of the operational environment. The GUIDE system is shown as a single logical unit, emphasizing its role as the central coordinator between users and external services.
End users interact with GUIDE by providing high-level travel preferences, including destination, duration, and category-based interests. Based on these inputs, the system generates structured itineraries and corresponding route plans. Routing and map-related computations are delegated to external services, namely OpenStreetMap (OSM) [12] for geographic data and the Open Source Routing Machine (OSRM) [13] for distance and route calculations.
In addition, GUIDE integrates external content and media services to enrich the user experience. Descriptive information about points of interest is retrieved and refined using publicly available knowledge sources such as Wikipedia [6]. Multilingual audio narration is generated via a text-to-speech (TTS) service [7], enabling users to access destination information in multiple languages.

2 Current software architecture

At the current stage (April 2026 - updated), the GUIDE system has been fully implemented in accordance with the architectural decisions defined in the High-Level Design (HLD) and refined in the Low-Level Design (LLD) documents [8]. The system is no longer conceptual; instead, it operates as a functional, end-to-end application consisting of a web-based client, a stateless backend service layer, and integrated routing and data components. The OSRM routing engine is deployed as an isolated Docker container and POI data is persisted in a Supabase (PostgreSQL) database accessed via the PostgREST interface.

The implemented architecture follows a hybrid layered and service-oriented approach, preserving the separation of concerns defined in the HLD. The main architectural layers (Presentation, API Boundary, Core Services, Integration, and Data Access) are realized as concrete modules in the codebase. This alignment ensures traceability between design artifacts and implementation, as also reflected in the LLD traceability matrix and package decomposition.

The **Presentation Layer** is implemented as a React-based single-page application. It handles user interaction, including trip parameter input, route visualization, POI exploration, and interactive replanning. Communication with the backend is performed through RESTful API calls, and map rendering is achieved using Leaflet with OpenStreetMap tiles [15, 11, 12].

The **API Boundary Layer** is implemented using a FastAPI-based backend [10]. This layer serves as the controlled entry point of the system, handling request validation, input normalization, and error mapping. Controllers such as RouteController and PoiController dispatch validated requests to the corresponding core services.

The **Core Services Layer** contains the main business logic of the system, including itinerary generation, POI filtering, routing orchestration, and content aggregation. Services such as ItineraryService, PoiService, RoutingService, and ContentService operate on validated inputs

and remain stateless, ensuring scalability and predictable behavior.

The **Integration Layer** encapsulates communication with the routing engine. A self-hosted OSRM instance is used for route computation, and all routing requests are handled through a dedicated client component. This design isolates external dependencies and prevents them from affecting core service logic.

The **Data Access Layer** provides structured access to POI metadata and multimedia content. The system uses a curated dataset containing POI information, images, and pre-generated audio assets. As defined in the LLD, data access is performed through repository abstractions, ensuring flexibility for future storage changes.

A key architectural decision implemented in the system is the separation between offline data preparation and runtime execution. External services such as content sources and text-to-speech providers are used only during dataset preparation. At runtime, the system relies entirely on self-hosted components, which improves reliability, reduces latency, and ensures deterministic behavior under load.

Overall, the implemented system closely follows the architecture proposed in the HLD and detailed in the LLD. The consistency between design and implementation validates the architectural decisions and demonstrates that the system can operate as a scalable, modular, and maintainable travel planning platform.

3 Proposed software architecture

3.1 Overview

The proposed architecture follows a hybrid Layered and Service-Oriented approach. Responsibilities are separated across presentation, application logic, data management, and integration layers to support scalability, maintainability, and clear separation of concerns.

The layered structure provides a systematic organization of system responsibilities, allowing each layer to evolve independently with minimal impact on others. Service-oriented boundaries are applied within and across layers to encapsulate core functionalities such as itinerary generation, routing orchestration, and content delivery.

This architectural approach enables the system to accommodate future growth, integrate additional services, and adapt to changing requirements while preserving architectural clarity and stability.

Architecture summary. The diagram in Figure.2 illustrates the major layers and core services of GUIDE. It highlights how user-facing components connect to the API gateway layer and how backend services collaborate with data storage and routing components.

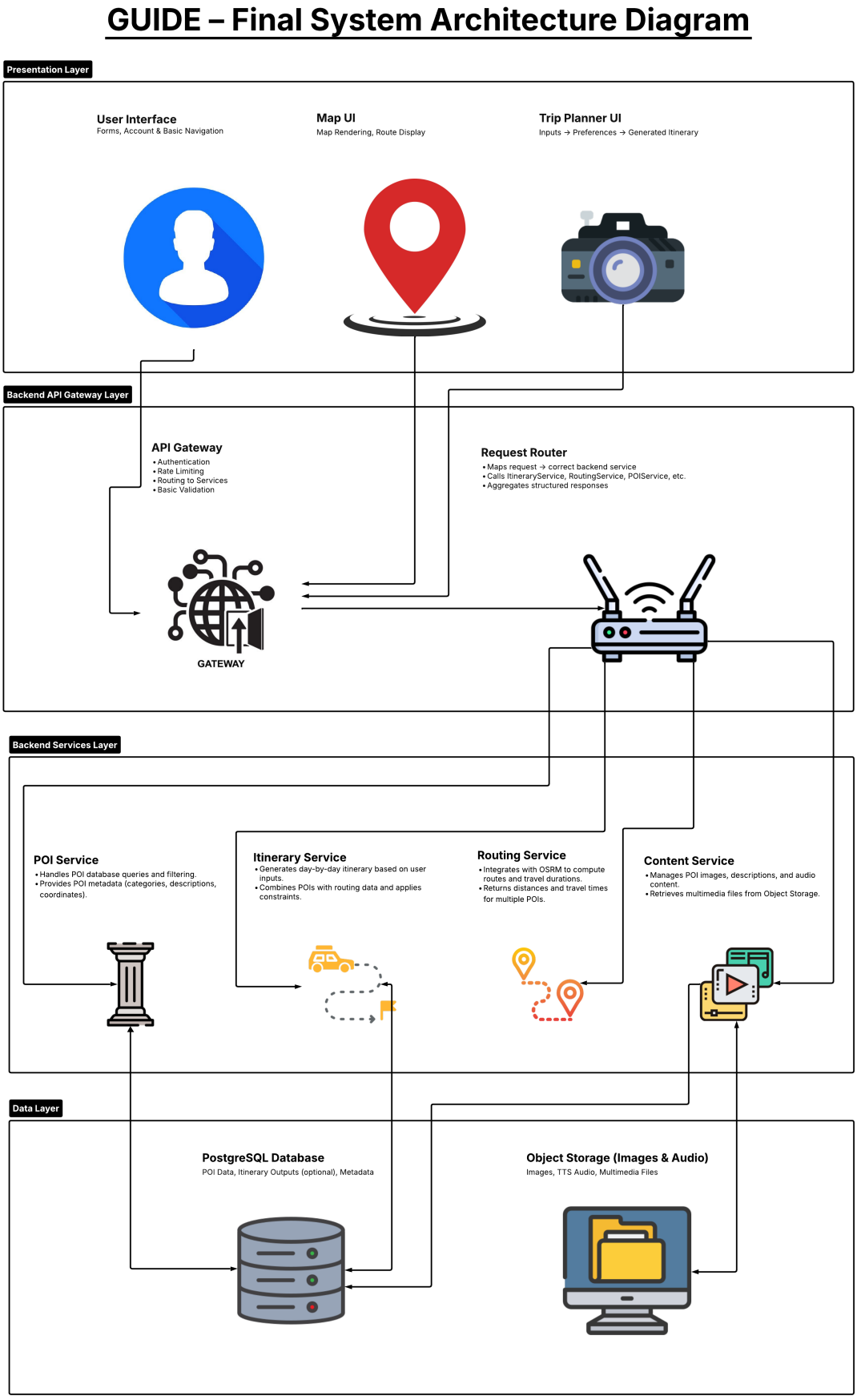

Figure 2: High-level architecture of GUIDE (layers and core services).

3.2 Subsystem decomposition

The system is composed of the following subsystems:

- **Presentation Layer:** Web-based UI components such as map visualization and itinerary display.
- **Backend API Gateway Layer:** Request validation, routing, and basic protection mechanisms such as request control and rate limiting.
- **Backend Services Layer:** Core services such as POI service, itinerary generation, routing orchestration, and content delivery.
- **Data Layer:** Persistent storage for POIs and optional outputs, plus media storage for images and audio assets.

Figure 3 provides a detailed visual representation of these subsystems, illustrating the service interactions, data flow, and the isolation of the Open Source Routing Machine (OSRM) engine within the architecture [13].

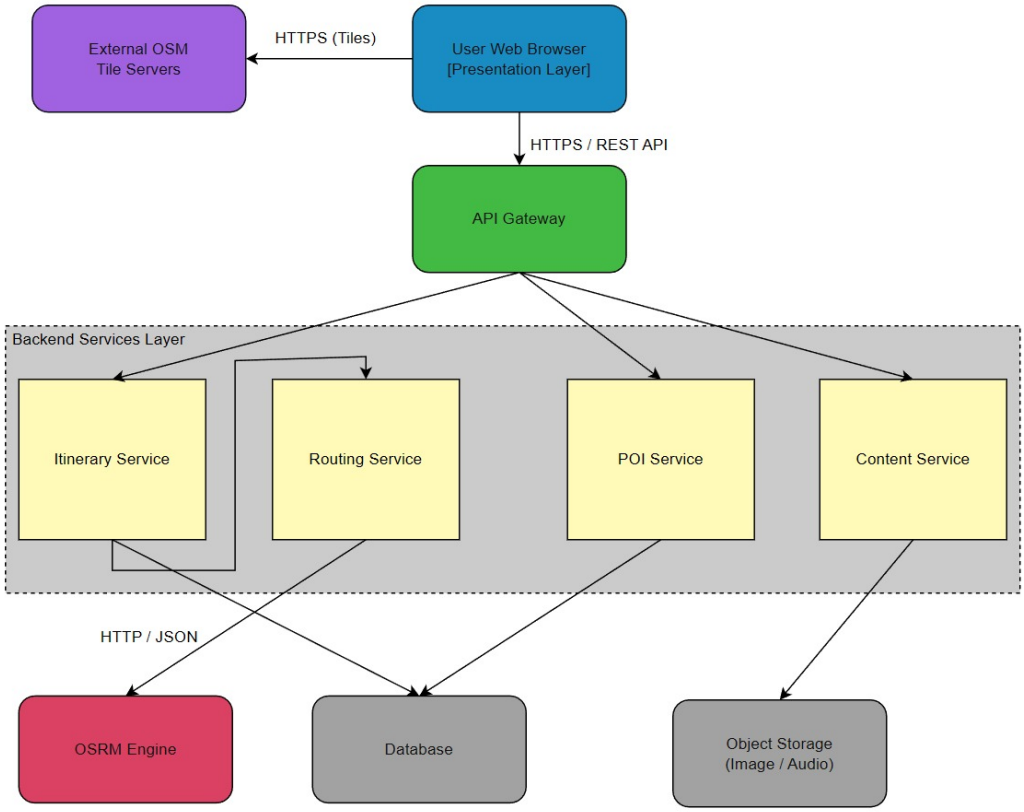

Figure 3: Detailed system block diagram showing service interactions and data flow.

Subsystem summary table. The following table summarizes the main subsystems, their responsibilities, and their key interfaces.

Subsystem	Primary responsibilities	Key interfaces
Presentation (Web UI)	Collects user inputs (city, duration, preferences), renders interactive maps and itinerary views, displays POI details, and triggers audio playback.	HTTPS REST calls to backend API, map tiles (OSM/Leaflet) [12], media retrieval endpoints.
Backend API Gateway	Authenticates/validates requests (where applicable), applies basic rate limiting, routes requests to internal services, and returns structured responses to the UI.	Public REST API endpoints, internal service routing, logging/monitoring hooks.
Backend Services (Core)	Implements business logic: POI filtering/selection, itinerary construction, routing orchestration, and content aggregation for responses.	Internal service-to-service calls, OSRM [13], data access layer.
Data Layer	Stores POI metadata and optional itinerary outputs; serves read-only static assets (images, pre-generated TTS audio, etc.). Supports migration path from JSON/SQLite to PostgreSQL/PostGIS.	Database interface, object/media storage access.

3.3 Hardware/software mapping

The deployment structure of GUIDE is shown in Figure 3. Client devices access the system via HTTPS. The backend runs as a stateless API service, interacts with an OSRM routing engine over an internal network interface, and accesses a database and media storage for POI metadata and static multimedia assets.

Deployment summary. The diagram in Figure.3 shows how GUIDE components are mapped onto physical nodes. It emphasizes the separation between runtime components and external APIs that are used for offline dataset preparation.

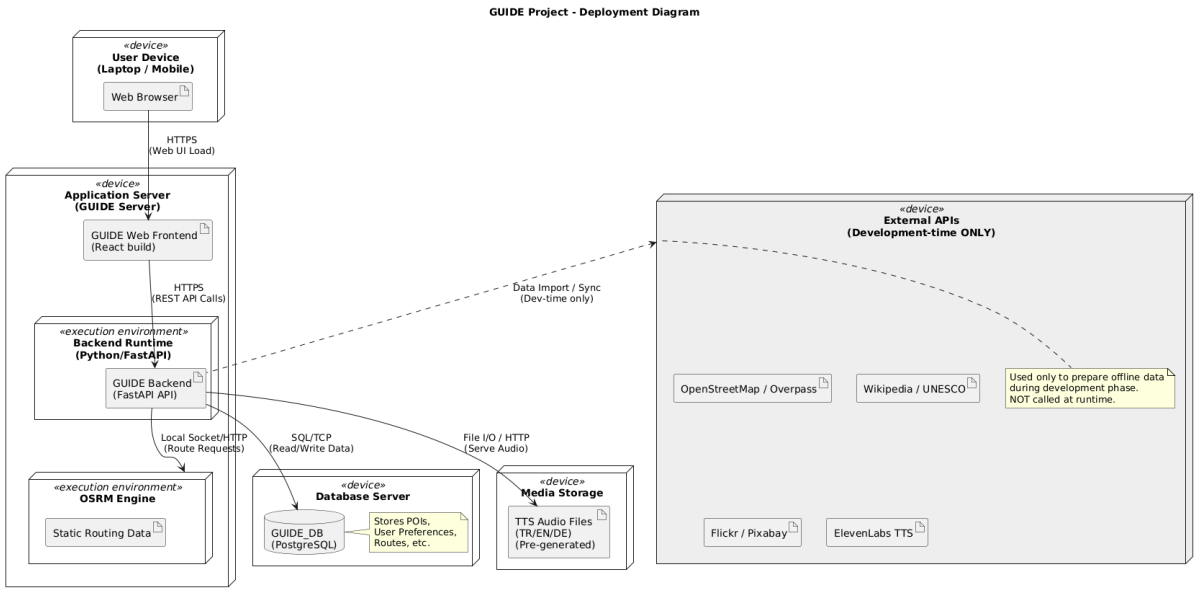

Figure 4: GUIDE deployment diagram.

Scalability and growth strategy. As user traffic increases, the GUIDE deployment can evolve from a single-node setup into a horizontally scalable multi-node topology. The backend is designed as a *stateless* API layer, enabling multiple identical application instances to run behind a load balancer. Static assets (images and pre-generated TTS audio) can be served from shared media storage or a CDN-like layer to reduce backend load. The routing tier (OSRM) is deployed as a separate service and can be scaled by running multiple OSRM instances behind an internal load balancer, isolating routing computation from the API tier. Importantly, external content APIs are used only during dataset preparation, so runtime scalability does not introduce third-party coupling; capacity planning remains focused on the application, routing, and storage layers that are fully under project control.

3.4 Persistent data management

Persistent data is managed via a centralized data store and local repositories. Data access is abstracted through service layers to ensure consistency and maintainability. Static multimedia (images and TTS audio) is stored separately and served as read-only assets during runtime.

Baseline vs growth path. For the implemented system, POI metadata, content, and media asset references are stored in a **Supabase**-managed PostgreSQL database, accessed through the PostgREST Data API. Static multimedia files (images and pre-generated audio) are stored as externally referenced assets and served via URL references rather than being embedded in API responses. This design keeps API payloads compact and delegates media delivery to the storage layer.

3.5 Access control and security

Security is enforced through controlled access to protected resources, secure communication channels (HTTPS/TLS), and isolation of sensitive components. The API gateway layer may also apply basic validation, rate limiting, and request throttling to protect backend services from abuse or sudden traffic increases.

3.6 Global software control

System control is distributed across services. Logging, configuration, and monitoring are treated as global concerns to avoid single points of failure and to support operational visibility.

3.7 Boundary conditions

The system accounts for varying user loads, partial subsystem failures, network latency, and external service unavailability. A key architectural boundary is the *runtime vs development-time* separation: third-party content APIs are used only during offline dataset preparation, while runtime operation relies on self-hosted components. This reduces dependency risks and keeps runtime behavior deterministic and controllable under different operational conditions.

4 Subsystem services

This section summarizes the major services provided by each subsystem at a high level. Service responsibilities are intentionally defined through clear boundaries to support maintainability, testability, and scalability.

4.1 Presentation services (Web UI)

The Presentation Subsystem provides user-facing services that enable interaction with GUIDE:

- **Preference input and validation (client-side):** Collects trip parameters such as city, duration, categories, and optional filters.
- **Map visualization:** Renders POIs and routes on an interactive map (e.g., Leaflet with OSM tiles) and supports zoom/pan/selection [11, 12].
- **Itinerary visualization:** Displays ordered POIs, day-by-day splits, and route summaries returned by the backend.
- **Media playback and POI details:** Shows descriptions and triggers the playback of pre-generated audio guides when available.
- **System explanation and flow guidance ("How GUIDE Works" view):** Presents a high-level, step-by-step overview of the system workflow, explaining how user inputs are transformed into an itinerary and routing outputs to improve transparency and user understanding.
- **Itinerary replanning and adjustment:** Enables users to revise previously generated itineraries by modifying preferences or constraints and triggers partial or full regeneration requests to the backend.

4.2 Application services

The Backend Services Layer provides the following core application services:

- **POI Filtering and Candidate Pool Service:** Selects candidate POIs based on user-defined destination, main categories, subcategories, and exclusion preferences. The service constructs a filtered POI pool that serves as the input for candidate route generation.
- **Candidate Route Generation and Evaluation Service:** Constructs and evaluates candidate routes from the filtered POI pool according to user preferences, daily route distance ranges, estimated visit durations, and route composition criteria. This service generates day-by-day itineraries by comparing alternative candidate routes and selecting feasible high-quality results.
- **Routing and Distance Estimation Service:** Estimates route distances during candidate generation using straight-line distance calculations within predefined distance ranges to reduce computational cost. For final selected routes, precise road-network distances and paths are computed using OSRM [13].
- **Content Delivery Service:** Aggregates POI metadata and media references such as images/audios and returns a response that fits the format of the UI.

4.3 Data services (POI & media storage)

The Data Layer provides data services as read-heavy operations:

- **POI Repository Service:** Read-only access to POI metadata and categories.
- **Media Asset Service:** Serves static multimedia assets efficiently without requiring external calls at runtime.

4.4 Security services (API protection and access control)

Security is enforced through a combination of transport security and API protection mechanisms:

- **Secure communication:** HTTPS/TLS between client and backend [14].
- **Request validation:** Input validation and schema checks at the API boundary.

4.5 Integration services (routing and offline preparation)

Integration services define how GUIDE interacts with internal and external systems:

- **OSRM integration service (runtime):** Communicates with self-hosted OSRM [13] through HTTP APIs and normalizes route responses for internal use.
- **Offline data preparation pipeline:** Collects POI descriptions, images, and audio content from third-party providers such as Wikipedia and Google services (Places API and Text-to-Speech) [6, 16, 17], and produces the static dataset used during runtime.

Service boundary note. A key design principle is that third-party content providers are excluded from runtime dependencies. Runtime service quality and availability are therefore primarily determined by self-hosted components (backend services, local data store, and OSRM), which improve reliability and support deterministic performance under load.

5 Glossary

- **API (Application Programming Interface):** A set of protocols and tools for building software applications, enabling communication between different software components.
- **CDN (Content Delivery Network):** A distributed system of servers used to deliver static content such as images, audio, or other media to users efficiently. In the context of GUIDE, a CDN-like mechanism refers to serving static assets from shared or replicated storage to reduce backend load and improve response times.
- **Context-Aware System:** A system that uses contextual information (location, time, preferences) to provide relevant services and recommendations.
- **GDPR (General Data Protection Regulation):** European Union regulation (EU 2016/679) governing data protection and privacy for individuals within the EU.
- **KVKK (Kişisel Verilerin Korunması Kanunu):** Turkish Personal Data Protection Law (Law No. 6698) regulating the processing and protection of personal data.
- **OSM (OpenStreetMap):** A collaborative project that creates a free, editable map of the world, used for geographical data and map visualization.
- **OSRM (Open Source Routing Machine):** A high-performance routing engine designed to find the shortest path in road networks, used for route calculation and navigation.
- **POI (Point of Interest):** A specific location that may be of interest to travelers, such as historical sites, museums, restaurants, or natural landmarks.
- **RESTful API:** An application programming interface that follows the REST architectural principles for web services communication.
- **Route Optimization:** The process of determining the most efficient route between multiple points, considering various constraints like distance, time, and preferences.
- **TTS (Text-to-Speech):** Technology that converts written text into spoken audio output.
- **UI (User Interface):** The visual elements and interactive components through which users interact with the system.
- **UX (User Experience):** The overall experience and satisfaction a user has when interacting with a product or system.

References

[1] IEEE Computer Society, *IEEE Std 1016-2009: Systems and Software Engineering—Software Design Descriptions*, 2009.

[2] ISO/IEC, *ISO/IEC 25010:2011 Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models*, 2011.

[3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and software engineering—Life cycle processes—Requirements engineering*, 2018.

[4] OpenStreetMap Foundation, "API v0.6 Documentation," [Online]. Available: `https://wiki.openstreetmap.org/wiki/API_v0.6`. [Accessed: Jan. 2026].

[5] Project OSRM, "Open Source Routing Machine HTTP API Documentation," Version 5.24, [Online]. Available: `https://project-osrm.org/docs/v5.24.0/api/`. [Accessed: Jan. 2026].

[6] Wikimedia Foundation, "MediaWiki Action API Documentation," [Online]. Available: `https://www.mediawiki.org/wiki/API:Main_page`. [Accessed: Jan. 2026].

[7] Google Cloud, *Text-to-Speech API*. Available at: `https://cloud.google.com/text-to-speech` (Accessed: 2026).

[8] GUIDE Project Team, *Low-Level Design Document*, TOBB University of Economics and Technology, Computer Engineering Department, 2026.

[9] Meta, *React: A JavaScript library for building user interfaces*, Available at: `https://react.dev/`

[10] S. Ramírez, *FastAPI Framework*, Available at: `https://fastapi.tiangolo.com/`

[11] V. Agafonkin, *Leaflet: An open-source JavaScript library for interactive maps*, Available at: `https://leafletjs.com/`

[12] OpenStreetMap Contributors, *OpenStreetMap*, Available at: `https://www.openstreetmap.org/`

[13] OSRM Project, *Open Source Routing Machine*, Available at: `https://project-osrm.org/`

[14] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, Internet Engineering Task Force (IETF), 2018.

[15] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral Dissertation, University of California, Irvine, 2000.

[16] Google, *Google Places API Documentation*, Available at: https://developers.google.com/maps/documentation/places

[17] Google, *Google Cloud Text-to-Speech API Documentation*, Available at: https://cloud.google.com/text-to-speech/docs